

Car Rental Blueprint

Your Car Rental System, condensed for revision: the strategy wiring, the domain model, the booking pipeline, and the traps you already hit once so you never hit them in an interview.

THE PATTERN

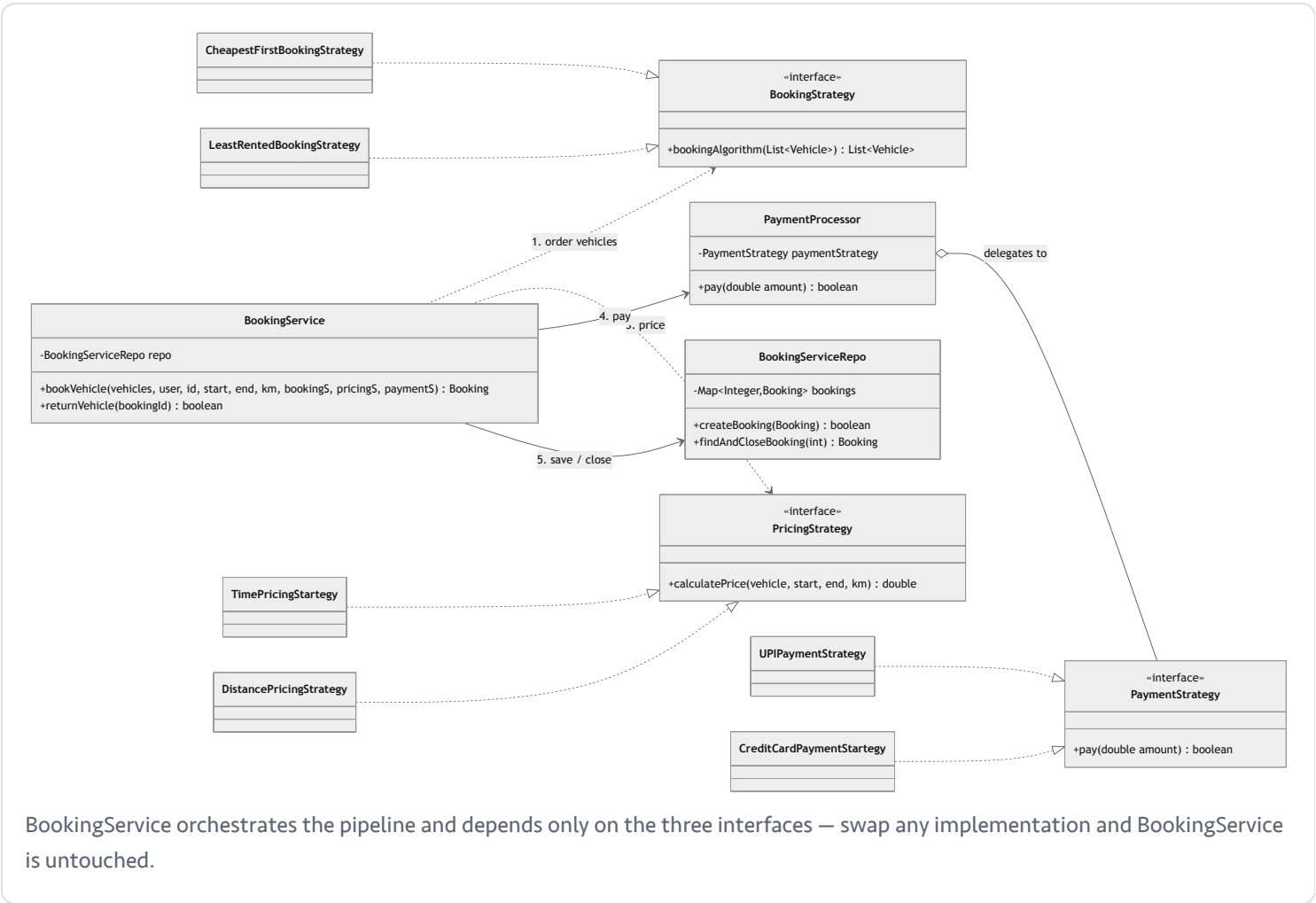
Strategy Pattern in one line

A family of interchangeable algorithms behind one interface, chosen by the caller at runtime. Each of your three strategies answers exactly one question in the booking pipeline — and `BookingService` never knows which concrete class it's talking to.

STRATEGY	QUESTION IT ANSWERS	IMPLEMENTATIONS
<code>BookingStrategy</code>	Which vehicle do I offer first?	<code>CheapestFirst</code> · <code>LeastRented</code>
<code>PricingStrategy</code>	How much does it cost?	<code>TimePricing</code> · <code>DistancePricing</code>
<code>PaymentStrategy</code>	How is it paid?	<code>UPI</code> · <code>CreditCard</code>

Strategy wiring — the heart of the design

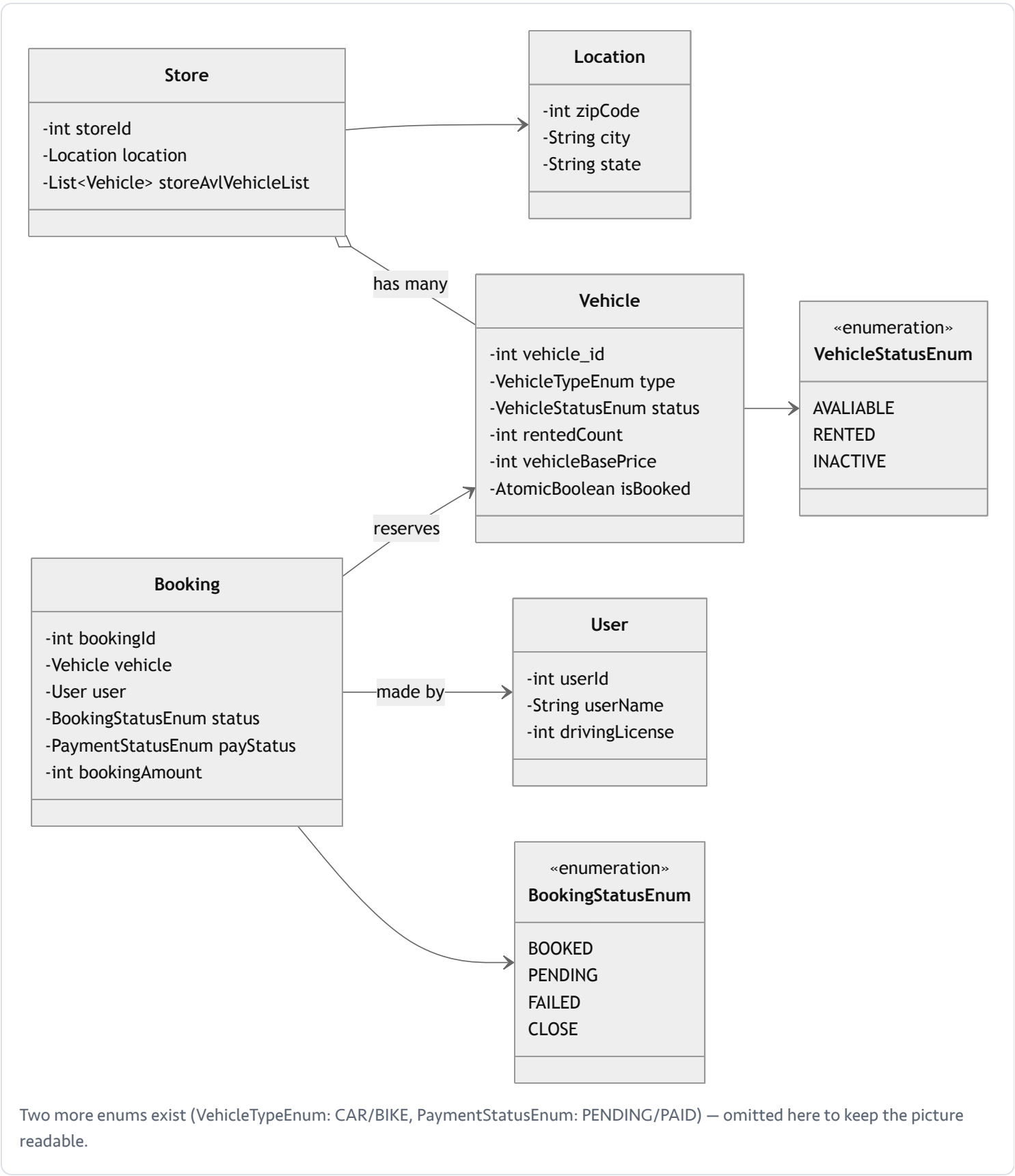
Solid arrows are "uses"; dashed hollow-head arrows are "implements". `PaymentProcessor` is the *context* class of the pattern: it holds a strategy and delegates to it.



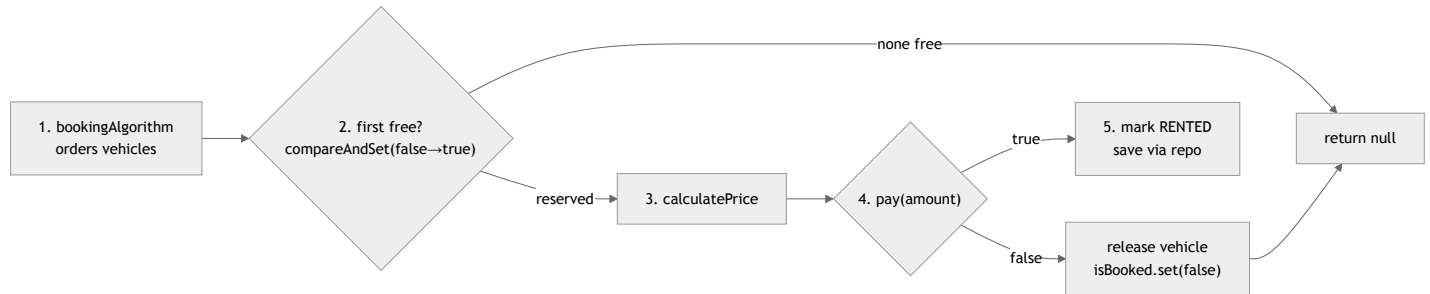
`BookingService` orchestrates the pipeline and depends only on the three interfaces — swap any implementation and `BookingService` is untouched.

Domain model

The entities the services move around. A diamond means "has a" (aggregation).



bookVehicle — the 5-step pipeline



The payment-failure branch must release the reservation — otherwise the vehicle is booked forever.

- 1 **Select** — `bookingStrategy.bookingAlgorithm(list)` orders the vehicles. The service doesn't know or care how.
- 2 **Reserve** — walk the list, take the first where `isBooked.compareAndSet(false, true)` returns `true`. The return value tells you whether *you* won the reservation.
- 3 **Price** — `pricingStrategy.calculatePrice(vehicle, start, end, km)`. Amount comes from a strategy, never hardcoded.
- 4 **Pay** — `new PaymentProcessor(paymentStrategy).pay(amount)`. If it returns `false`, release the vehicle and bail.
- 5 **Persist** — mark `RENTED`, bump `rentedCount`, create the `Booking`, save via `BookingServiceRepo`.

`returnVehicle(id)` is the reverse: repo closes and returns the `Booking` → set vehicle back to `AVAILABLE` → `isBooked.set(false)`.

GOTCHAS

Traps you already hit once

Money is `double`, not `boolean`

An interface method must carry real inputs and outputs — `pay(double amount)` returning `boolean`. A no-arg `void` method can't participate in any decision.

Return the strategy's result

`PaymentProcessor.pay` must return `paymentStrategy.pay(amount)`. Hardcoding `true` means a failed payment can never be detected.

Initialize `AtomicBoolean`

`= new AtomicBoolean(false)` at declaration — a declared-but-null field throws `NullPointerException` on first use.

Check the CAS return value

The whole point of `compareAndSet` is the boolean it returns: `true` = you got the car, `false` = someone else did. Calling it and ignoring the result defeats thread-safety.

Undo on failure

Reserved the vehicle but payment failed? `isBooked.set(false)` before returning, or the car leaks — stuck booked forever.

No — on `LocalDateTime`

Use `Duration.between(start, end).toHours()`. Java has no operator overloading for dates.

INTERVIEW LINES

Say this out loud

- "I can add a `PayPalPaymentStrategy` without touching `BookingService` — that's the **Open/Closed Principle**: open for extension, closed for modification."
- "`BookingService` depends on the **interfaces**, not the concrete classes — that's **Dependency Inversion**."
- "`PaymentProcessor` is the **context** class of the Strategy pattern — it holds a strategy reference and delegates."
- "`AtomicBoolean.compareAndSet` makes the vehicle reservation **thread-safe** — two concurrent users can't book the same car, because only one CAS wins."
- "The repo isolates **storage** from **orchestration** — swap the `HashMap` for a database and the service logic doesn't change."